

Name:		Index Number:		Class:	
-------	--	---------------	--	--------	--



DUNMAN HIGH SCHOOL

Preliminary Examination

Year 6

COMPUTING (Higher 2)

9569/02

Paper 2 (Lab-based)

24 August 2026

3 hours

Additional Materials: Insert

Electronic version of `plaintext.txt` file

Electronic version of `publickey.txt` file

Electronic version of `privatekey.txt` file

Electronic version of `CLIENT.py` file

Electronic version of `prelim_results.json` file

READ THESE INSTRUCTIONS FIRST

Write your name, index number and class on the work you hand in.

Write in dark blue or black pen on both sides of the paper.

You may use an HB pencil for any diagrams or graphs.

Do not use staples, paper clips, glue or correction fluid.

Answer **all** the questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

The number of marks is given in brackets [] at the end of each question or part question. The total number of marks for this paper is 100.

Instructions to candidates:

Your program code and output for Tasks 1 to 3 should be saved in a single `.ipynb` file each using Jupyter Notebook. For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

Make sure that each of your `.ipynb` files shows the required output in Jupyter Notebook.

- 1 The Rivest-Shamir-Adleman (RSA) algorithm is the industrial standard for asymmetric encryption. Its benefit is it removes the need for the sender and receiver to exchange encryption keys. The sender encrypts the message with the receiver's public key, made readily available over the internet. After the message is transmitted, all the receiver needs to do is to decrypt using the receiver's private key.

Using a simplified RSA algorithm, plaintext data in source file `plaintext.txt` will be encrypted using the public key, then decrypted using a separate private key. Both keys are stored in source files `publickey.txt` and `privatekey.txt`, and the math linking them has been abstracted to keep things simple.

The public and private keys are defined as tuples (e, n) and (d, n) respectively, where e is the public exponent, d the private exponent and n is the modulus. Note that all calculations are on ASCII values.

The encryption formula is:

$$\text{encrypted_char} = \text{char}^e \bmod n$$

To decrypt:

$$\text{decrypted_char} = \text{encrypted_char}^d \bmod n$$

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol `#` to indicate the sub-task the program code belongs to, for example:

```
In [ ]: #Task 1.1
        Program code
```

Output:

Task 1.1

The text files `publickey.txt` and `privatekey.txt` each contain a key made up of 2 numbers separated by a comma.

Write program code for the function `load_key()` that takes a filename as a parameter, reads the file and returns the key as a tuple of integers. [3]

Task 1.2

The text file `plaintext.txt` contains unencrypted string data.

Write program code for the function `load_message()` that takes a filename as a parameter and returns the string contents of the file. [2]

Task 1.3

Write program code for the function `encrypt_message()` that takes `message` string, public exponent integer `e` and modulus integer `n` as three parameters. It performs RSA encryption on each character in `message` and returns a list of integer values. [5]

Task 1.4

Write program code for the function `decrypt_message()` that takes a list of encrypted integer values, private exponent integer `d` and modulus integer `n` as three parameters. It performs RSA decryption on each value in the list and returns a string of decrypted characters. [5]

Task 1.5

Write program code to:

1. call the function `load_key()` on `publickey.txt` and `privatekey.txt` to load the public and private keys
2. call the function `load_message()` on `plaintext.txt` to load the plaintext message to be encrypted
3. call the function `encrypt_message()` to encrypt the message
4. call the function `decrypt_message()` to decrypt the message

Output the results in every step. [4]

Save your Jupyter Notebook for Task 1.

- 2** A Simple File Storage Scheme called **SFSS** manages the allocation of computer storage nodes when creating and deleting files in a computer system. It uses a free space pointer to keep track of the free space list, a dictionary as a file directory and a fixed size array to store the nodes. It can allocate free nodes to create new files and return unused nodes back to the free space list when files are deleted. Each file is thus a linked list populated by nodes kept inside an array, with the filename and starting node stored in the dictionary. The free space list is also a linked list maintained by a free space pointer, which is set to -1 if it is empty.

A node is initialised with empty string as data and pointer as -1 . The array is initialised to contain a fixed number of unused nodes, with the pointer of each node initialised to point to the next node, except the last node which terminates the free space list with pointer kept as -1 . The free space pointer is initialised to point to the first node of the array.

When a new file needs to be created, **SFSS** will assign and link a suitable number of nodes to store data. Each node can only store one character of data. Subsequently, it will create a key-value pair in the dictionary with filename as key and the starting node of the file data as value. For example, when 'file1' containing data 'ace' and 'file2' containing 'man' are added, in that order, into a 10 node **SFSS** instance, the directory, free space pointer and array look like this:

File Directory: {'file1': 0, 'file2': 3}

Free space pointer: 6

Array State:

Index	Data	Next
0	a	1
1	c	2
2	e	-1
3	m	4
4	a	5
5	n	-1
6		7
7		8
8		9
9		-1

When a file is deleted, **SFSS** will return the nodes freed up by the file deletion by traversing the file's linked list from the front, linking each node **to the front of the free space list**, until the end of file is reached. For better visualisation, the data in the unused node is then overwritten with an empty string. For example, when 'file1' is deleted, the directory, free space pointer and array look like this:

File Directory: {'file2': 3}

Free space pointer: 2

Array State:

Index	Data	Next
0		6
1		0
2		1
3	m	4
4	a	5
5	n	-1
6		7
7		8
8		9
9		-1

Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [ ]: #Task 2.1
        Program code
```

Output:

Task 2.1

The class `Node` contains two attributes:

- `data`: the node's single character data, initialised as an empty string
- `pointer`: the node's next pointer, initialized to -1

The class `Node` has the following methods:

- a constructor to initialise the two attributes
- `set_data()` that assigns a value to `data`
- `set_pointer()` that assigns a value to `pointer`
- `get_data()` that returns `data`
- `get_pointer()` that returns `pointer`

Write program code to declare the class `Node` and its methods.

[3]

Task 2.2

The class `SFSS` contains three attributes:

- `array`: a list initialised to contain `n` linked instances of the `Node` class
- `directory`: stores filename and starting node, initialised as an empty dictionary
- `free`: the free space list pointer, initialised to point to first node in array

The class `SFSS` has the following methods:

- A constructor that takes in an integer parameter `n` and initialises the three attributes
- `set_free()` that assigns a value to `free`
- `get_free()` that returns `free`
- `get_array()` that returns `array`
- `get_directory()` that returns `directory`
- `save_file()` takes in string parameters `filename` and `datastring` and does the following:
 1. Outputs an appropriate error message if free space list is empty and exits. If the free space list is non-empty, it is assumed that there are sufficient nodes to store the new file data.

2. Iterates through the `datastring`, retrieves nodes needed from the free space list and stores the respective characters from `datastring` into the nodes
 3. Creates a new dictionary key-value pair in directory once all data is stored in nodes
- `delete_file()` takes in string parameter `filename` and does the following:
 1. Outputs an appropriate error message if `filename` does not exist in directory and exits
 2. Iterates through the file's linked list nodes, sets data for the deleted node to empty string and returns it to the free space list
 3. Once all the nodes from deleted file are added back to free space list, its key-value pair is deleted from directory
 - `display()` prints the file directory, free space pointer and the index, data and pointer of the nodes in the array. The output of a newly initialised instance of SFSS class of size 3 may look like this:

```
File Directory: {}
Free space pointer: 0
Array State:
Index | Data | Next
  0 |   | 1
  1 |   | 2
  2 |   | -1
```

Write program code to declare the class `SFSS` and its methods.

[23]

Task 2.3

Test your program with the following steps:

1. Initialise an instance of `SFSS` class of size 10
2. Save a new file named 'file1' with data 'ace'
3. Save a new file named 'file2' with data 'man'
4. Delete file named 'file1'
5. Save a new file named 'file3' with data 'nudhigh'
6. Run `display()` method and show contents of the `SFSS` instance

Your final output should look like this:

```
File Directory: {'file2': 3, 'file3': 2}
Free space pointer: -1
Array State:
Index | Data | Next
  0 | d | 6
  1 | u | 0
  2 | n | 1
  3 | m | 4
  4 | a | 5
  5 | n | -1
  6 | h | 7
  7 | i | 8
  8 | g | 9
  9 | h | -1
```

Show the output for each step.

[4]

Save your Jupyter Notebook for Task 2.

- 3 You created a minesweeper game where the server program host the game for a human player (client program). The minesweeper board consists of 16 coordinates (4 rows & 4 columns). At the start of the game, 10 of the coordinates are **randomly** chosen to contain a mine. The aim of the game is to find the remaining 6 coordinates that do not have a mine.

When a coordinate is chosen by the human player, the server program will check if it contains a mine. If it does, the game is over and the human player has lost. If the chosen coordinate does not contain a mine, the server program will inform the client program on the number of adjacent mines around that selected coordinate. For example, coordinate (a) has 7 adjacent mines, coordinate (b) has 4 adjacent mines and coordinate (c) has 3 adjacent mines.

y-coordinate	4	MINE	MINE	MINE	(c)
	3	MINE	(a)	MINE	MINE
	2		MINE	MINE	MINE
	1		MINE	(b)	
		1	2	3	4
		x-coordinate			

Name your Jupyter Notebook as:

TASK3_<your name>_<centre number>_<index number>.ipynb

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [ ]: #Task 3.1
        Program code
```

Output:

Task 3.1

The data of the 16 coordinates are stored in a relational database (minesweeper.db). The database has one table called Board.

Board (xCoordinate, yCoordinate, hasMine, checked)

Board:

- xCoordinate – the x coordinate value, ranging from 1 to 4 (both inclusive)
- yCoordinate – the y coordinate value, ranging from 1 to 4 (both inclusive)
- hasMine – to indicate if the particular coordinate has a mine (value 1) or not (value 0) [Note: In each game there are 10 coordinates that have a mine (value 1) and 6 coordinates that do not have a mine (value 0)]
- checked – to indicate if the particular coordinate has been checked by the user (value 1) or not (value 0) [default value at the start of game is 0]

Write a Python program that uses SQL code to create the above database. Define the primary key for Board table.

[4]

Task 3.2

Study the sample program outputs in the next three pages to determine your code design, output format and socket protocol. User inputs are underlined and bold. Take note that the mine locations are different for each run of the program.

Sample Server Program 1:

```
-----
MINESWEEPER GAME [SERVER]
-----

Waiting for player to join
-----
User x coordinate: 3, User y coordinate: 2
no_of_surrounding_mines: 6
no_of_user_moves_made: 1

User x coordinate: 4, User y coordinate: 1
Hit MINE
-----
GAME ENDED
-----
```

Sample Client Program 1:

```
-----
MINESWEEPER GAME [CLIENT]
-----

Select your coordinate:
INPUT x Coordinate: 3
INPUT y Coordinate: 2

Great job! You did not hit a mine!
```

There are 6 mine(s) around your last selected coordinate and you have checked 1 coordinate(s)!

Select your coordinate:

INPUT x Coordinate: 4

INPUT y Coordinate: 1

You have hit a mine! You lose!

GAME ENDED

Sample Server Program 2:

MINESWEEPER GAME [SERVER]

Waiting for player to join

User x coordinate: 1, User y coordinate: 3

no_of_surrounding_mines: 4

no_of_coordinates_checked: 1

User x coordinate: 2, User y coordinate: 2

no_of_surrounding_mines: 4

no_of_coordinates_checked: 2

User x coordinate: 3, User y coordinate: 1

no_of_surrounding_mines: 2

no_of_coordinates_checked: 3

User x coordinate: 3, User y coordinate: 2

no_of_surrounding_mines: 4

no_of_coordinates_checked: 4

User x coordinate: 3, User y coordinate: 3

no_of_surrounding_mines: 5

no_of_coordinates_checked: 5

User x coordinate: 4, User y coordinate: 2

no_of_surrounding_mines: 2

no_of_coordinates_checked: 6

GAME ENDED

Sample Client Program 2:

MINESWEEPER GAME [CLIENT]

Select your coordinate:

INPUT x Coordinate: 1

INPUT y Coordinate: 3

```

Great job! You did not hit a mine!
There are 4 mine(s) around your last selected coordinate and you
have checked 1 coordinate(s)!

Select your coordinate:
INPUT x Coordinate: 2
INPUT y Coordinate: 2

Great job! You did not hit a mine!
There are 4 mine(s) around your last selected coordinate and you
have checked 2 coordinate(s)!

Select your coordinate:
INPUT x Coordinate: 3
INPUT y Coordinate: 1

Great job! You did not hit a mine!
There are 2 mine(s) around your last selected coordinate and you
have checked 3 coordinate(s)!

Select your coordinate:
INPUT x Coordinate: 3
INPUT y Coordinate: 2

Great job! You did not hit a mine!
There are 4 mine(s) around your last selected coordinate and you
have checked 4 coordinate(s)!

Select your coordinate:
INPUT x Coordinate: 3
INPUT y Coordinate: 3

Great job! You did not hit a mine!
There are 5 mine(s) around your last selected coordinate and you
have checked 5 coordinate(s)!

Select your coordinate:
INPUT x Coordinate: 4
INPUT y Coordinate: 2

Great job! You did not hit a mine!
You have found all the coordinates without mine. You win!
-----
GAME ENDED
-----

```

Using socket programming, code the server program to:

1. Establish connection to `minesweeper.db` database.
2. Delete all records in `Board` table.
3. Insert 16 records for the 16 coordinates in `Board` table. [Note: Exactly 10 coordinates must be randomly chosen to have a mine (`hasMine` value of 1) and 6 coordinates to not have a mine (`hasMine` value of 0) and the default value of `checked` is 0 for all records at the start of the game.]
4. Wait for connection from client program.
5. Collect user input for the x coordinate and y coordinate. [Note: You can assume that the user inputs given are always valid.]
6. Update the `checked` value of the selected coordinate to 1.
7. Check if the selected coordinate has a mine (`hasMine` value of 1).
8. If the selected coordinate has a mine, send the message "MINE" to client program and end the server program.
9. If the selected coordinate does not have a mine, send the message to client program on the number of surrounding mines with respect to the selected coordinate and the total number of coordinates the user has checked.
10. Repeat Steps 5 to 9 until user has checked a total of 6 coordinates and end the server program. [Note: only 6 of the coordinates do not have a mine.]

You are provided with the completed client program, `CLIENT.py`. Code the corresponding server program as described above and name it as `SERVER_<your name>_<centre number>_<index number>.py` [21]

Save all your files for Task 3.

- 4 Dunman High School's Data Analytics Committee intends to develop a Web App for viewing Year 6 students' Prelim Examination results. The json file `prelim_results.json` contains values of each student's ID, full name and the prelim grades of their respective subjects. The data is to be inserted into a **NoSQL** database DHS under the collection `Prelims`. Note that all data in the `Prelims` collection should be deleted at the launch of the Web App and repopulate the collection with data in the current `prelim_results.json` file in the directory. The sample of the Web App is displayed below:



Student ID	Student Name	Subject 1	Grade	Subject 2	Grade	Subject 3	Grade	Subject 4	Grade	Subject 5	Grade
S10001	Aiden Tan Wei Ming	H1 GP	A	H2 MATH	A	H2 CHEM	A	H2 PHY	A	H2 ECONS	A
S10002	Priya Nair Suresh	H1 GP	B	H2 BIO	A	H2 CHEM	A	H2 MATH	A	H1 ECONS	A
S10003	Muhammad Hafiz bin Roslan	H1 GP	C	H2 HIST	B	H2 ECONS	A	H2 GEO	B	H1 MATH	C
S10004	Chloe Lim Xin Yi	H1 GP	A	H2 ECONS	A	H2 MATH	A	H2 COMP	A	H2 PHY	A
S10005	Rajan Pillai Krishnan	H1 GP	D	H2 BIO	C	H2 CHEM	C	H2 MATH	D	H1 PHY	C
S10006	Natasha Wong Hui Ling	H1 GP	A	H2 ELIT	A	H2 HIST	A	H2 GEO	A	H2 ECONS	A
S10007	Darren Ong Boon Kiat	H1 GP	B	H2 COMP	A	H2 MATH	B	H2 PHY	A	H1 CHEM	B
S10008	Siti Hajar binte Azman	H1 GP	A	H2 BIO	B	H2 CHEM	A	H2 ECONS	B	H1 MATH	A
S10009	Ethan Goh Jun Hao	H1 GP	D	H2 HIST	C	H2 ELIT	C	H2 GEO	C	H1 ECONS	D
S10010	Isabelle Cheong Mei Xuan	H1 GP	A	H2 BIO	A	H2 CHEM	A	H2 MATH	A	H2 PHY	A
S10011	Kevin Yeo Zhi Xiang	H1 GP	C	H2 COMP	B	H2 MATH	C	H2 ECONS	B	H1 BIO	C
S10012	Amelia Raj Kumari	H1 GP	A	H2 ECONS	A	H2 HIST	A	H2 ELIT	A	H2 GEO	A
S10013	Brandon Koh Wei Jie	H1 GP	B	H2 PHY	B	H2 MATH	A	H2 COMP	A	H1 CHEM	B
S10014	Nurul Izzah binte Hamid	H1 GP	C	H2 GEO	C	H2 HIST	C	H2 ECONS	C	H1 ELIT	D
S10015	Lucas Sim Jian Wei	H1 GP	A	H2 MATH	A	H2 PHY	A	H2 COMP	A	H2 CHEM	A
S10016	Jasmine Teo Hui Min	H1 GP	A	H2 BIO	A	H2 CHEM	B	H2 ECONS	A	H1 GEO	A
S10017	Farid bin Sulaiman	H1 GP	E	H2 MATH	D	H2 PHY	E	H2 CHEM	D	H1 BIO	E
S10018	Rachel Ng Shu Fen	H1 GP	A	H2 ELIT	A	H2 HIST	A	H2 ECONS	A	H1 GEO	A
S10019	Joel Lau Cheng Hao	H1 GP	B	H2 ECONS	B	H2 MATH	B	H2 GEO	B	H1 HIST	B
S10020	Mei Ling Tan Xiu Ying	H1 GP	C	H2 BIO	C	H2 CHEM	B	H2 MATH	C	H2 PHY	C
S10021	Alicia Phua Jing Wen	H1 GP	A	H2 HIST	A	H2 ELIT	A	H2 GEO	A	H1 ECONS	A
S10022	Zachary Ho Wen Bin	H1 GP	B	H2 COMP	A	H2 MATH	A	H2 PHY	B	H2 ECONS	A
S10023	Divya Menon Lakshmi	H1 GP	A	H2 BIO	A	H2 CHEM	A	H2 ECONS	A	H1 MATH	A
S10024	Marcus Chew Jia Jun	H1 GP	D	H2 GEO	C	H2 HIST	C	H2 ECONS	C	H1 ELIT	C
S10025	Hannah Foo Shi Qi	H1 GP	A	H2 MATH	A	H2 CHEM	A	H2 BIO	A	H2 ECONS	A
S10026	Syafiq bin Nordin	H1 GP	C	H2 PHY	C	H2 MATH	B	H2 COMP	B	H1 CHEM	C
S10027	Vanessa Kwan Pei Shan	H1 GP	A	H2 ECONS	A	H2 HIST	A	H2 GEO	A	H2 ELIT	A
S10028	Timothy Ang Beng Huat	H1 GP	B	H2 MATH	B	H2 PHY	B	H2 CHEM	B	H1 BIO	B
S10029	Nur Sabrina binte Ismail	H1 GP	A	H2 BIO	A	H2 ECONS	A	H2 GEO	A	H1 CHEM	A
S10030	Elijah Tan Rui Feng	H1 GP	C	H2 COMP	B	H2 MATH	C	H2 PHY	C	H1 ECONS	D

There is a dropdown menu available for the user to filter the results of an individual student. The dropdown menu contains all the full names of the students in the Prelims collection. Upon form submission, only the details of the selected student will be shown in the html table. The sample of the dropdown menu is displayed below:

Filter Student:

Student ID	Student Name	Subject 1	Grade	Subject 2	Grade	Subject 3	Grade	Subject 4	Grade	Subject 5	Grade
S10001	Muhammad Hafiz bin Roslan	H1 GP	A	H2 MATH	A	H2 CHEM	A	H2 PHY	A	H2 ECONS	A
S10002	Chloe Lim Xin Yi	H1 GP	B	H2 BIO	A	H2 CHEM	A	H2 MATH	A	H1 ECONS	A
S10003	Rajan Pillai Krishnan	H1 GP	C	H2 HIST	B	H2 ECONS	A	H2 GEO	B	H1 MATH	C
S10004	Natasha Wong Hui Ling	H1 GP	A	H2 ECONS	A	H2 MATH	A	H2 COMP	A	H2 PHY	A
S10005	Darren Ong Boon Kiat	H1 GP	D	H2 BIO	C	H2 CHEM	C	H2 MATH	D	H1 PHY	C
S10006	Siti Hajar binte Azman	H1 GP	A	H2 ELIT	A	H2 HIST	A	H2 GEO	A	H2 ECONS	A
S10007	Ethan Goh Jun Hao	H1 GP	B	H2 COMP	A	H2 MATH	B	H2 PHY	A	H1 CHEM	B
S10008	Isabelle Cheong Mei Xuan	H1 GP	A	H2 BIO	B	H2 CHEM	A	H2 ECONS	B	H1 MATH	A
S10009	Kevin Yeo Zhi Xiang	H1 GP	D	H2 HIST	C	H2 ELIT	C	H2 GEO	C	H1 ECONS	D
S10010	Amelia Raj Kumari	H1 GP	A	H2 BIO	A	H2 CHEM	A	H2 MATH	A	H2 PHY	A
S10011	Brandon Koh Wei Jie	H1 GP	C	H2 COMP	B	H2 MATH	C	H2 ECONS	B	H1 BIO	C
S10012	Nurul Izzah binte Hamid	H1 GP	A	H2 ECONS	A	H2 HIST	A	H2 ELIT	A	H2 GEO	A
S10013	Lucas Sim Jian Wei	H1 GP	B	H2 PHY	B	H2 MATH	A	H2 COMP	A	H1 CHEM	B
S10014	Jasmine Teo Hui Min	H1 GP	C	H2 GEO	C	H2 HIST	C	H2 ECONS	C	H1 ELIT	D
S10015	Farid bin Sulaiman	H1 GP	A	H2 MATH	A	H2 PHY	A	H2 COMP	A	H2 CHEM	A
S10016	Rachel Ng Shu Fen	H1 GP	A	H2 BIO	A	H2 CHEM	B	H2 ECONS	A	H1 GEO	A
S10017	Joel Lau Cheng Hao	H1 GP	E	H2 MATH	D	H2 PHY	E	H2 CHEM	D	H1 BIO	E
S10018	Mei Ling Tan Xiu Ying	H1 GP	A	H2 ELIT	A	H2 HIST	A	H2 ECONS	A	H1 GEO	A
S10019	Joel Lau Cheng Hao	H1 GP	B	H2 ECONS	B	H2 MATH	B	H2 GEO	B	H1 HIST	B
S10020	Mei Ling Tan Xiu Ying	H1 GP	C	H2 BIO	C	H2 CHEM	B	H2 MATH	C	H2 PHY	C
S10021	Alicia Phua Jing Wen	H1 GP	A	H2 HIST	A	H2 ELIT	A	H2 GEO	A	H1 ECONS	A
S10022	Zachary Ho Wen Bin	H1 GP	B	H2 COMP	A	H2 MATH	A	H2 PHY	B	H2 ECONS	A

Below shows the web page when Ethan Goh Jun Hao is selected.

Filter Student:

Student ID	Student Name	Subject 1	Grade	Subject 2	Grade	Subject 3	Grade	Subject 4	Grade	Subject 5	Grade
S10009	Ethan Goh Jun Hao	H1 GP	D	H2 HIST	C	H2 ELIT	C	H2 GEO	C	H1 ECONS	D

Hint: Sample HTML code for dropdown menu:

```
<select name="language">
  <option value="gp">General Paper</option>
  <option value="ki">Knowledge Inquiry</option>
</select>
```

Task 4.1

Code the Python program, NoSQL Database and other necessary files to create the above web application.

Save your Python program as:

`Task_4_<your name>_<centre number>_<index number>.py`

with any additional files/subfolders in a folder named:

`Task_4_web_<your name>_<centre number>_<index number>`

Run the web application.

[25]

Filter the web page to only show Ethan Goh Jun Hao's details and save the output as:

`Task_4_<your name>_<centre number>_<index number>.html` [1]

Save all your files for Task 4.

End of Paper

